



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-1 机器启动

- 0. 如何维护你的实验仓库
- 1. 进入main函数
- 2. 串口与格式化输出
- 3. 共享资源与自旋锁
- 4. 实验小结

0. 如何维护你的实验仓库

分支名

master

lab-1

lab-2

lab-3

提交新的内容

每次实验的新增代码

继承而来的陈旧代码

git checkout -b 做的事情

最终状态

实验仓库包

括10个分支，

master + lab-

1 到 lab-9，

每个分支的

README不

同，但代码

是继承关系

分支管理相关：

1. `git branch -a` (查看分支信息)
2. `git checkout 分支名` (切换到某个分支)
3. `git checkout -b 分支名` (基于当前分支创建一个新分支并切换过去)

当你完成lab-1，开始做lab-2时，你应该在lab-1分支下执行 `git checkout -b lab-2`

向线上仓库提交代码的四个基本步骤：

1. 基于vscode在本地完成代码修改或增删
2. `git add`. (把所有修改放到暂存区)
3. `git commit -s -m “你想记录的提交信息”` (把暂存区存入本地仓库)
4. `git push` (将本地仓库同步到线上)

建议：如果你当天修改了代码，在当天工作结束时`git push`一次即可

1. 进入main函数

平时编写C语言程序时，容易产生这样的认知：只要定义了main函数，就一定会从它开始执行。

大家是否思考过：为什么？怎么做到的？我们不展开讲，你可以去查查。

很遗憾地告诉你：在编写OS内核这个“不一般”的C语言程序时，这种假设是不成立的。

于是，你遇到了第一个难题：对于OS内核来说，怎么进入一切逻辑的起点main函数呢？

1. QEMU的工作：

模拟条件下的QEMU会完成真实条件下的BIOS工作，
并将PC指针设置为一个约定好的地址 0x80000000。

2. kernel.ld的工作：

kernel.ld定义了kernel目录下多个.c和.S文件编译产物的
链接规则，它指定kernel-qemu.elf这个巨大程序的入口是_entry，同时各个section从0x80000000开始排布。

```
kernel.ld
1  OUTPUT_ARCH( "riscv" )
2  ENTRY( _entry )
3
4  SECTIONS
5  {
6      . = 0x80000000;
7
8      .text : {
9          *(.text .text.*)
10         . = ALIGN(0x1000);
11         /*
12         _trampoline = .;
13         *(trampsec)
14         . = ALIGN(0x1000);
15         ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
16         */
17         PROVIDE(etext = .);
18     }
19
```

```
# QEMU模拟器配置
QEMU = qemu-system-riscv64 # 指定QEMU程序
QEMUOPTS = -machine virt -bios none -kernel $(TARGET)/kernel/kernel-qemu.elf # 基础启动参数
QEMUOPTS += -m 128M -smp $(CPUNUM) -nographic # 内存、CPU数量及无图形界面配置
```

```
# 链接生成内核ELF文件
$(ELFKernel): $(KernelOBJ) $(UserOBJ)
$(LD) $(LDFLAGS) -T $(KERNEL_LD) $^ -o $@
```

1. 进入main函数

经过QEMU和kernel.ld的协作，我们可以将问题推进到这一步：

QEMU先完成BIOS的工作，然后将PC指针设置为0x80000000，对应OS内核编译链接后形成的kernel-qemu.elf中_entry标记的地址。

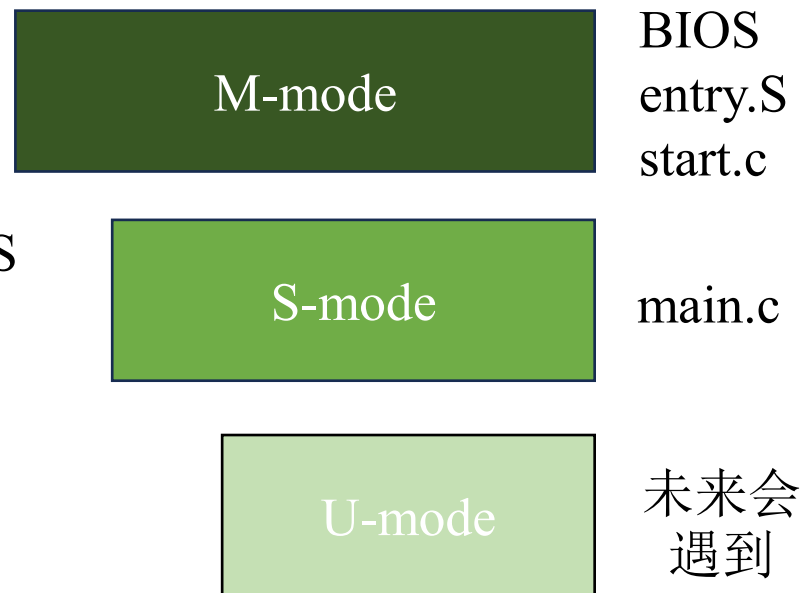
1. _entry标记位于entry.S文件，entry.S的作用是：

- 为函数栈栈顶指针设置合适的值 $SP = CPU_statck + (hartid + 1) * 4KB$
- 设置好SP后就可以调用start函数了

2. start函数位于start.c文件，start.c的作用是：

- 做一些M-mode寄存器的读写操作
- 让RISC-V的陷阱机制认为之前的状态是S-mode，以实现状态迁移
- 触发状态迁移并进入main函数(S-mode)

3. main函数的作用是系统和资源的初始化，未来会增加调度器启动。



RISC-V的三种模式（从上到下权力逐渐缩小）： OS内核启动阶段大部分时候处于M-mode，进入main函数后切换到S-mode，此后大部分时间都处于S-mode，直到用户态程序的诞生。

2. 串口与格式化输出

BIOS→**_entry**→**start**→**main** 历经千辛万苦终于进入了main函数后，

很快你就会发现新的问题：我怎么证明我真的进入main函数了呢？

思路1：我用GDB做逐行的执行流追踪！当然可以，但是不直观.....

思路2：打印一个经典的“Hello world!” 呗！怎么做到？

用户的视角是：字符通过键盘输入，屏幕输出

操作系统的视角是：通过串口设备（UART）来读写数据

控制外部设备的程序被称为“驱动程序”，驱动程序是如何工作的呢？

我们知道，可以通过读写设备内部的寄存器来感知和控制设备。

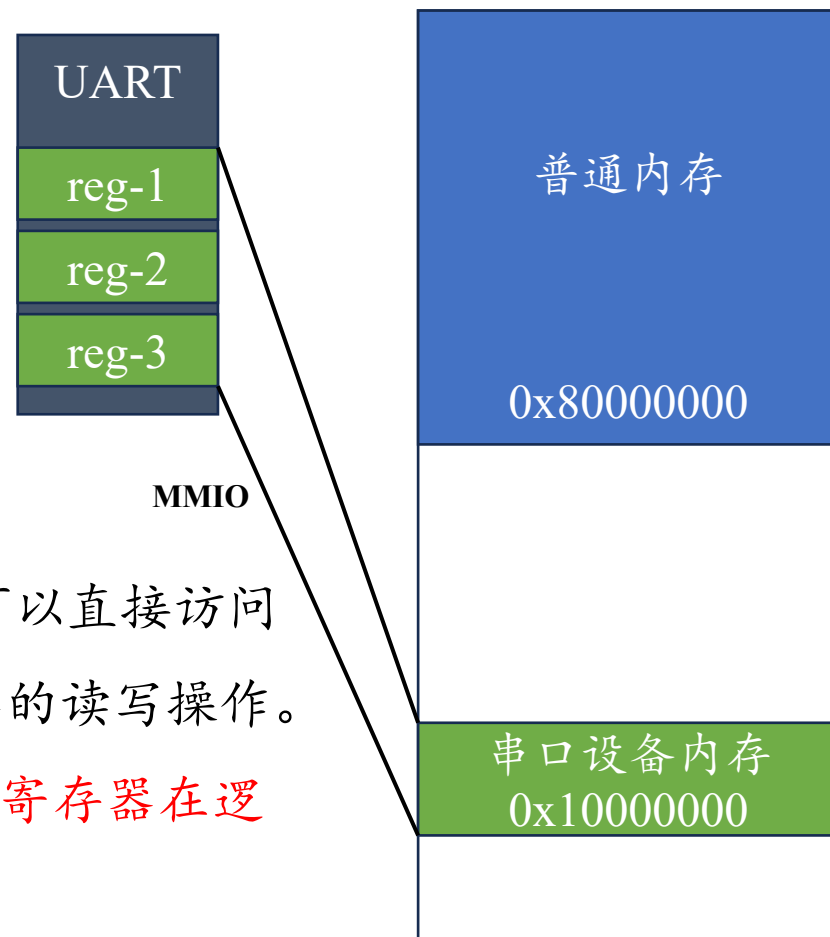
那么驱动程序如何读写设备内部的寄存器呢？

BIOS阶段会感知所有外部设备并将它们的寄存器“映射”到驱动程序可以直接访问的内存空间（MMIO），对这块内存空间的读写会被转换为对设备寄存器的读写操作。

综上所述，逻辑链条是：驱动程序可以读写内存→某块内存区域与设备寄存器在逻辑上被绑定→读写操作被传导至设备寄存器，实现设备控制。

```
// 读写寄存器的宏定义
#define Reg(reg) ((volatile unsigned char *) (UART_BASE + reg))
#define ReadReg(reg) (*(Reg(reg)))
#define WriteReg(reg, v) (*(Reg(reg)) = (v))

// UART 相关
#define UART_BASE 0x10000000ul
#define UART_IRQ 10
```



2. 串口与格式化输出

加入UART这种基本的输入输出外设后，OS内核具备了字符级的输入输出能力。

这只是个开始，接下来要基于UART实现格式化输出能力以适应变量输出的需求。

printf函数的特点是可以输出未知的变量，它至少需要支持下面五种类型：

```
/*
  标准化输出， 需要支持：
  1. %d (32位有符号数, 以10进制输出)
  2. %p (32位无符号数, 以16进制输出)
  3. %x (64位无符号数, 以0x开头的16进制输出)
  4. %c (单个字符)
  5. %s (字符串)
  提示：stdarg.h中的va_list中包括你需要的参数地址
*/
```

printf的声明没有明确列出传入参数的数量和类型，但是我们知道各个参数是在函数栈的特定位置连续放置的。因此，我们可以利用stdarg.h的va_list类型来获得各个参数。

之前不是说OS内核不会调用任何外部库吗？如果你仔细了解过stdarg.h，你就会发现它的本质是对函数栈的指令级读写操作，可以被编译为汇编指令，属于RISC-V体系结构基本能力的一部分。因此，我们可以安全地使用它而不依赖任何外部环境。

3. 共享资源与自旋锁

正常来说，①进入main函数和②格式化输出都已经实现了，lab-1的任务就完成了。

然而，OS内核需要支持多个CPU并行工作，因此不得不考虑资源共享和竞争的问题。

UART是我们遇到的第一种共享的设备资源，printf则是典型的“占有和使用资源”操作。

观察：printf通常由多个基于UART的字符输出操作构成，而且可以被中断。

例子：CPU-1要执行*printf*("Hello,world!"), CPU-2要执行*printf*("Hello, os!"), 竞争共享资源。

混乱的情况

hellohello,,world!os!

hheellllloo,,wosrld!!

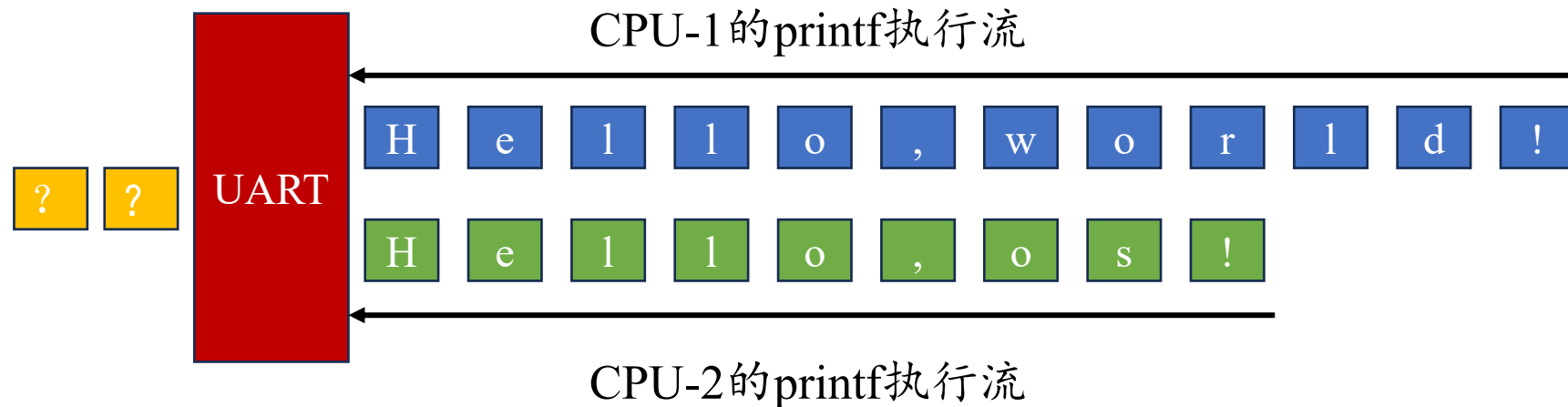
hhello,world!ello,os!

.....

有序的情况

hello,world!hello,os!

hello,os!hello,world!



OS内核需要一种同步机制来协调共享资源的有序使用

3. 共享资源与自旋锁

生活中也有类似的资源共享：很多人在公共卫生间排队上厕所。

如何保证有序性？进入卫生间的人会把门锁起来保证在一段时间内独占“马桶”这一资源。

类似的，OS内核里也需要“锁”来保证CPU中的执行流对某种共享资源的短期独占。

自旋锁是一种最根本最常见的锁，它的可靠性来自“开关中断”和“原子操作”。

引入锁机制后，共享资源的申请、占用、释放以锁作为边界：

在printf中使用自旋锁的方法

```
spinlock_t lk;
```

锁的初始化

```
spinlock_init(&lk, "print_lk");
```

上锁

```
spinlock_acquire(&lk);
```

独占资源

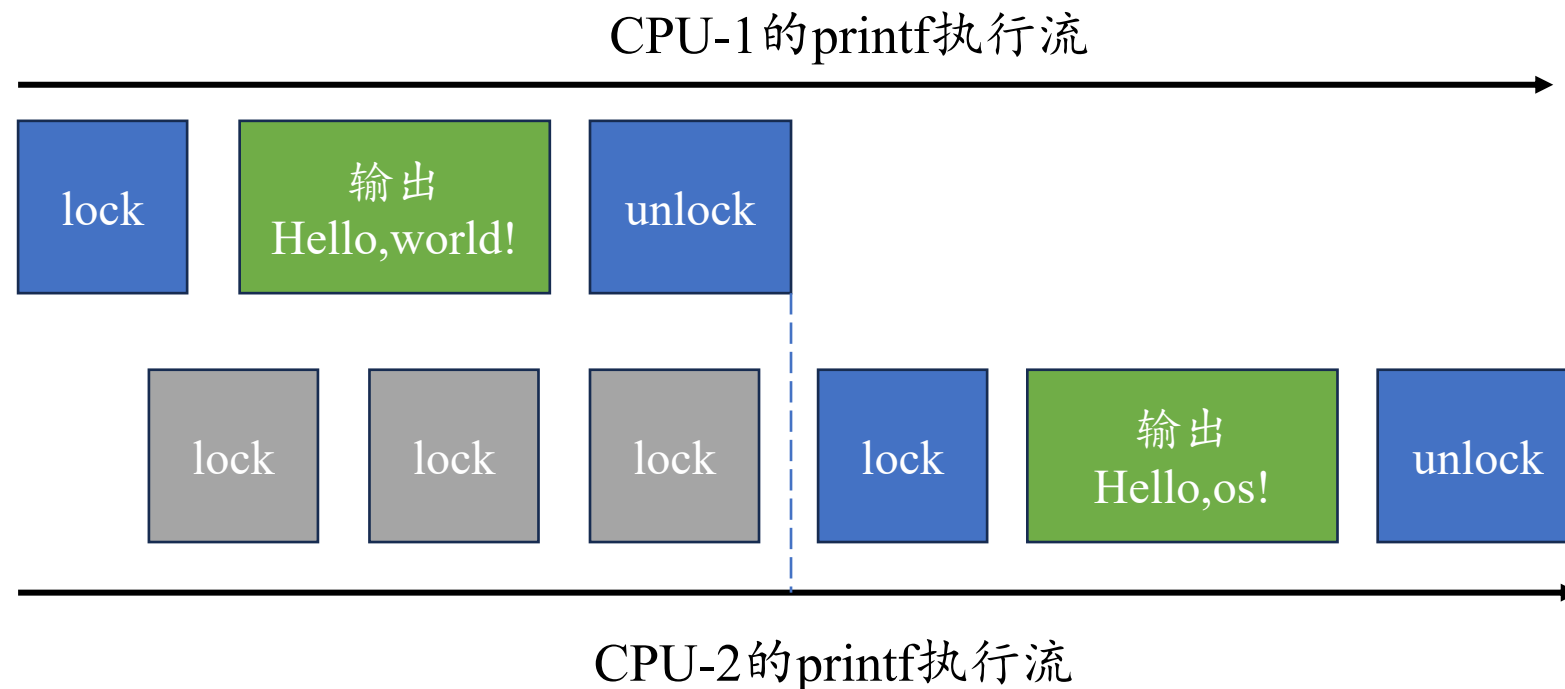
```
uart_putc_sync();
```

```
uart_putc_sync();
```

```
.....
```

解锁

```
spinlock_release(&lk);
```



4. 实验小结

